Using R Programming

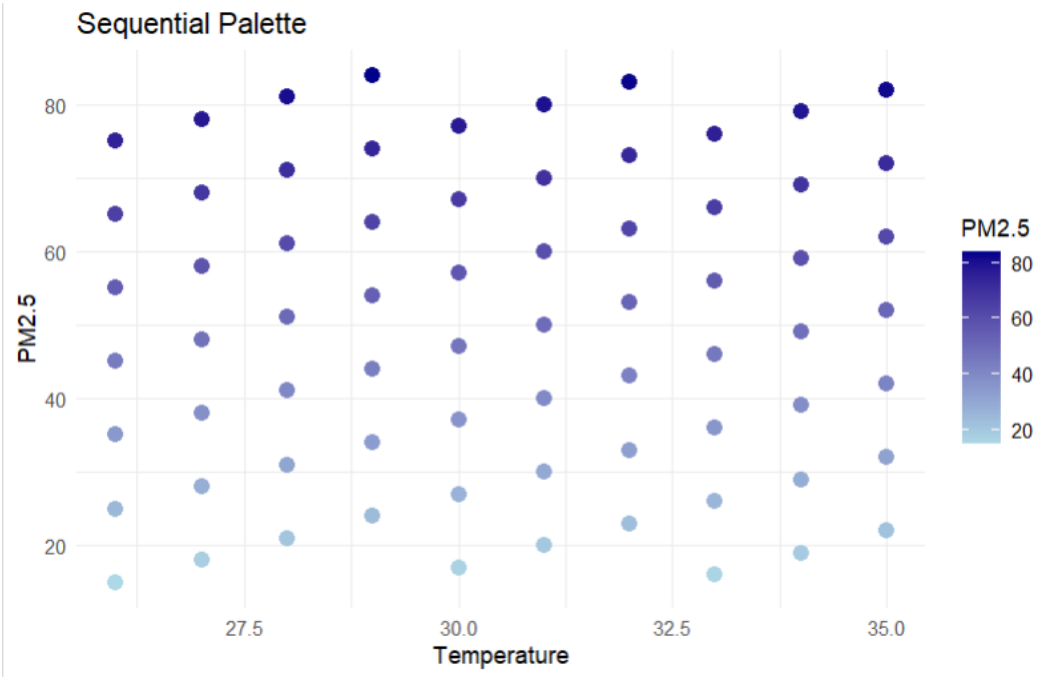
1. **Smart City Air Quality Monitoring:**

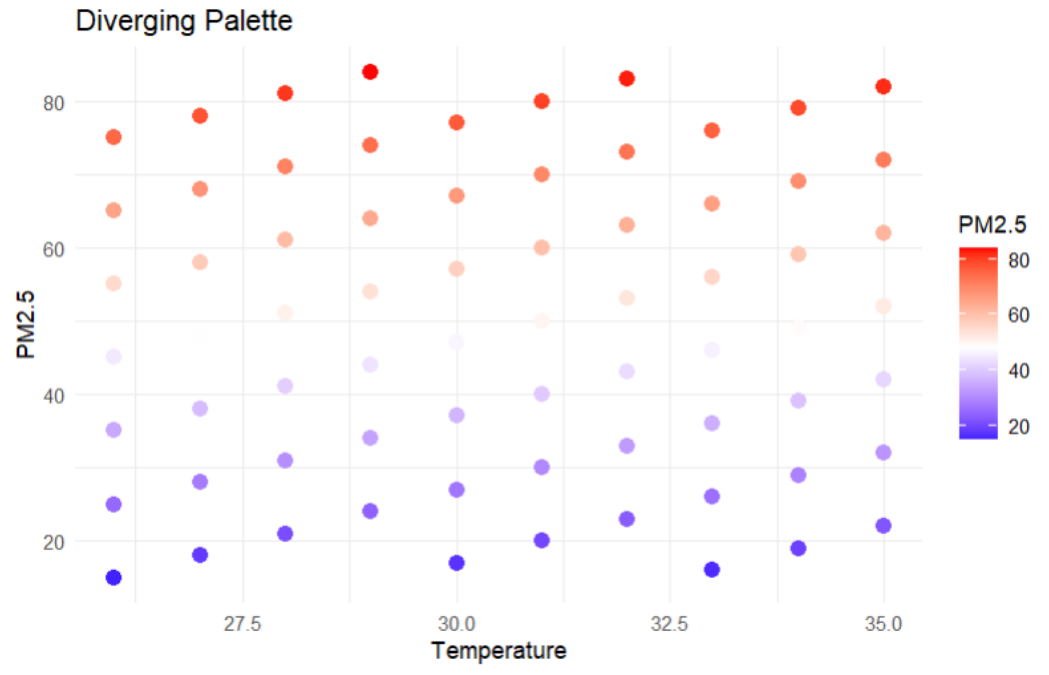
   Code :

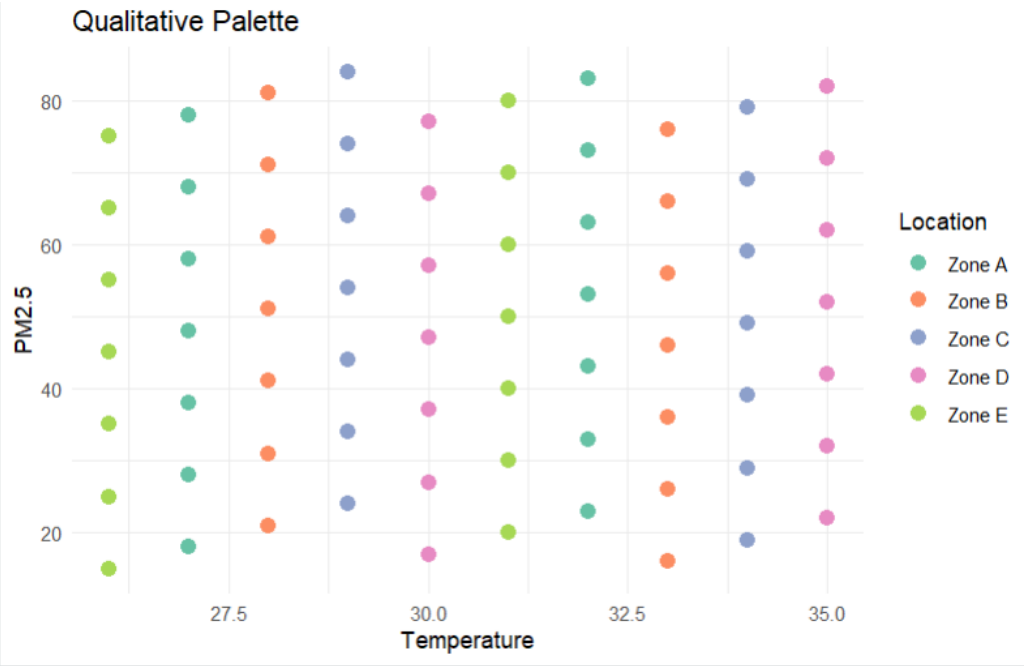
```
 library(ggplot2)

df <-
read.csv("C:/Users/naras/Downloads/DSA0602/air_quality_monitoring_150_sensors.
csv")

ggplot(df, aes(x = Temperature, y = PM2.5, color = PM2.5)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "lightblue", high = "darkblue") +
  labs(
    title = "Sequential Palette",
    x = "Temperature",
    y = "PM2.5"
  ) +
  theme_minimal()

ggplot(df, aes(x = Temperature, y = PM2.5, color = PM2.5)) +
  geom_point(size = 3) +
  scale_color_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = mean(df$PM2.5)
  ) +
  labs(
    title = "Diverging Palette",
    x = "Temperature",
    y = "PM2.5"
  ) +
  theme_minimal()

ggplot(df, aes(x = Temperature, y = PM2.5, color = Location)) +
  geom_point(size = 3) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "Qualitative Palette",
    x = "Temperature",
```

```
  y = "PM2.5"
) +
theme_minimal()
```

Output :







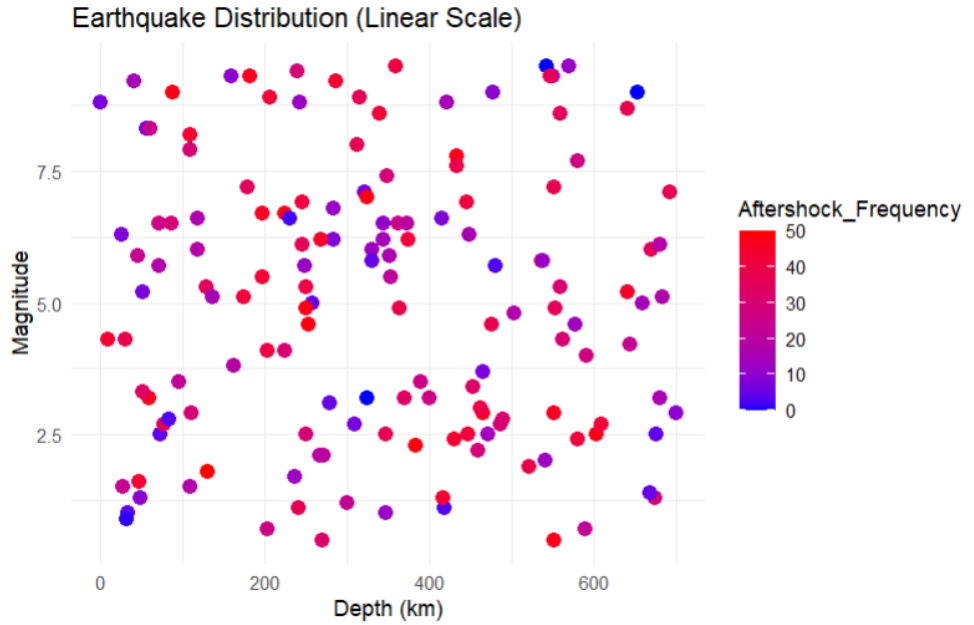
2. **Earthquake Risk Analysis:**

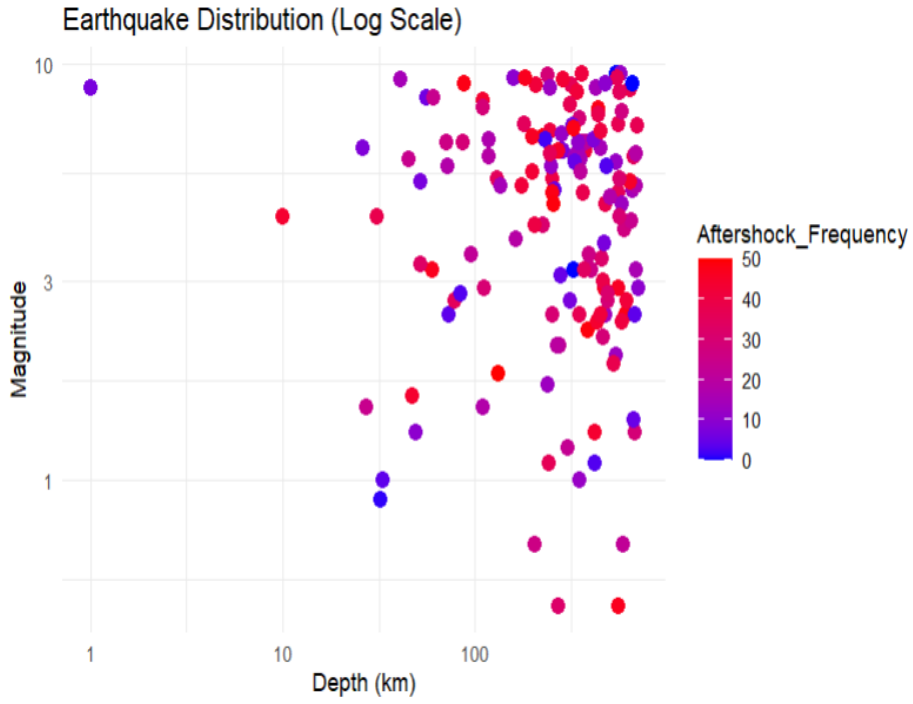
Code :

```
library(ggplot2)

df <- read.csv("C:/Users/naras/Downloads/earthquake_risk_analysis(2).csv")

ggplot(df, aes(x = Depth, y = Magnitude, color = Aftershock_Frequency)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "blue", high = "red") +
  labs(
    title = "Earthquake Distribution (Linear Scale)",
    x = "Depth (km)",
    y = "Magnitude"
  ) +
  theme_minimal()

ggplot(df, aes(x = Depth, y = Magnitude, color = Aftershock_Frequency)) +
  geom_point(size = 3) +
  scale_x_log10() +
  scale_y_log10() +
```

```
  scale_color_gradient(low = "blue", high = "red") +
  labs(
    title = "Earthquake Distribution (Log Scale)",
    x = "Depth (km)",
    y = "Magnitude"
  ) +
  theme_minimal()
```

Output :



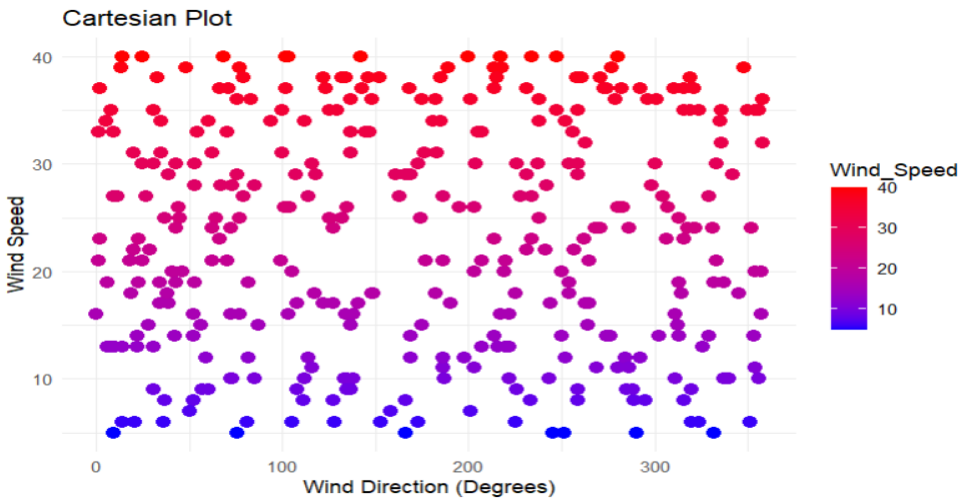


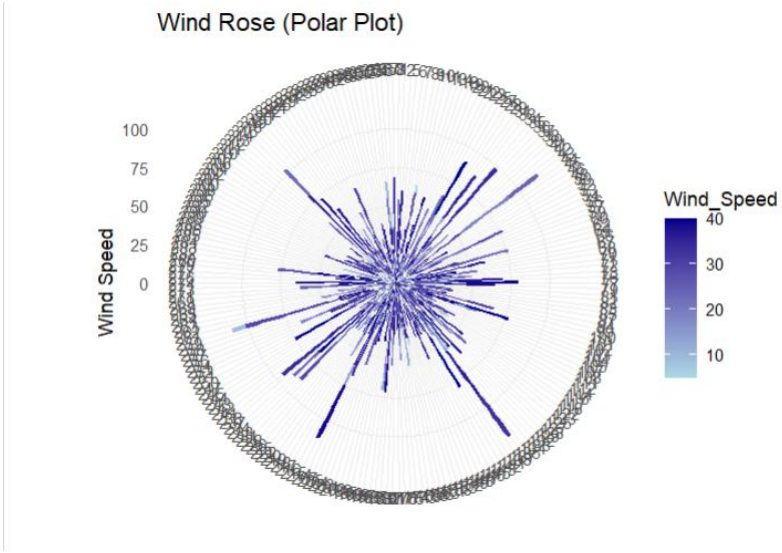
3. Wind Direction Analysis for Airport Operations:

Code :

```
library(ggplot2)

df <- read.csv("C:/Users/naras/Downloads/DSA0602/wind_direction_analysis.csv")

ggplot(df, aes(x = Wind_Direction, y = Wind_Speed, color = Wind_Speed)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "blue", high = "red") +
  labs(
    title = "Cartesian Plot",
    x = "Wind Direction (Degrees)",
    y = "Wind Speed"
  ) +
  theme_minimal()

ggplot(df, aes(x = factor(Wind_Direction), y = Wind_Speed, fill = Wind_Speed)) +
  geom_bar(stat = "identity") +
  coord_polar() +
  scale_fill_gradient(low = "lightblue", high = "darkblue") +
  labs(
    title = "Wind Rose (Polar Plot)",
    x = "",
    y = "Wind Speed"
  ) +
  theme_minimal()
```

Output :





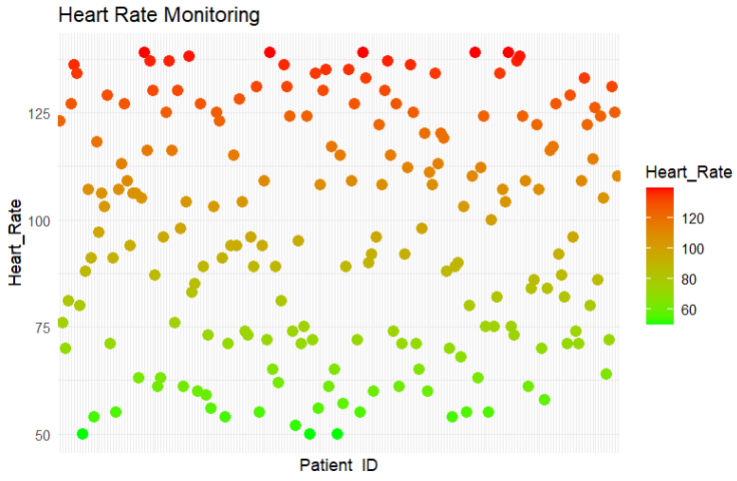
4. **Hospital Patient Monitoring Dashboard:**

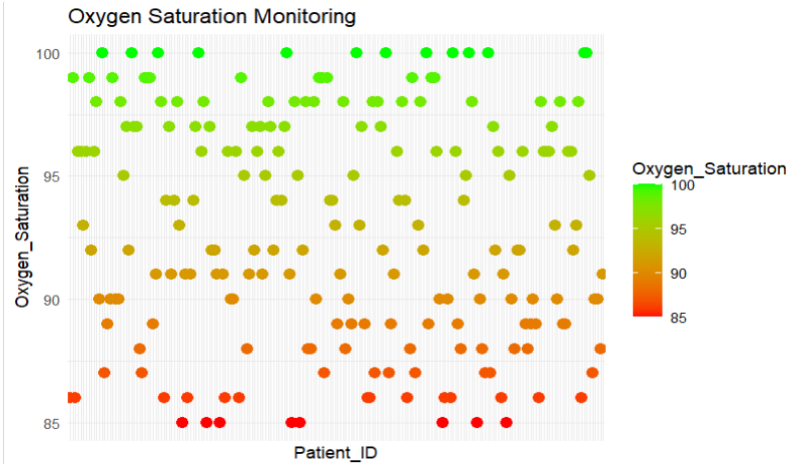
Code :

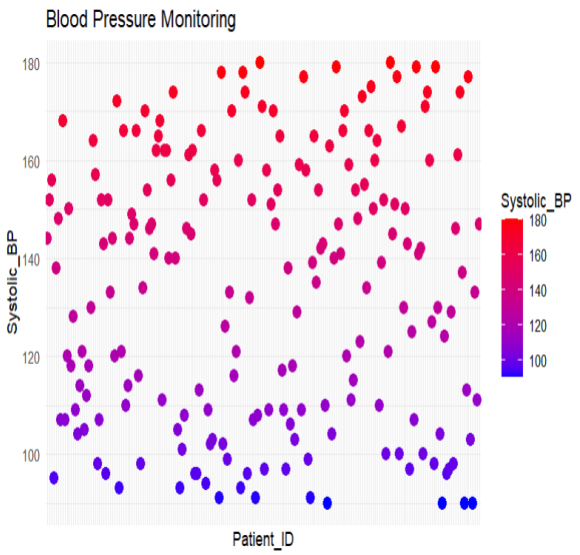
```
library(ggplot2)

df <-
read.csv("C:/Users/naras/Downloads/DSA0602/hospital_patient_monitoring_dashboa
rd.csv")

ggplot(df, aes(Patient_ID, Heart_Rate, color = Heart_Rate)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "green", high = "red") +
  theme_minimal() +
  theme(axis.text.x = element_blank()) +
  labs(title = "Heart Rate Monitoring")

ggplot(df, aes(Patient_ID, Oxygen_Saturation, color = Oxygen_Saturation)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "red", high = "green") +
  theme_minimal() +
  theme(axis.text.x = element_blank()) +
  labs(title = "Oxygen Saturation Monitoring")

ggplot(df, aes(Patient_ID, Systolic_BP, color = Systolic_BP)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "blue", high = "red") +
  theme_minimal() +
  theme(axis.text.x = element_blank()) +
```

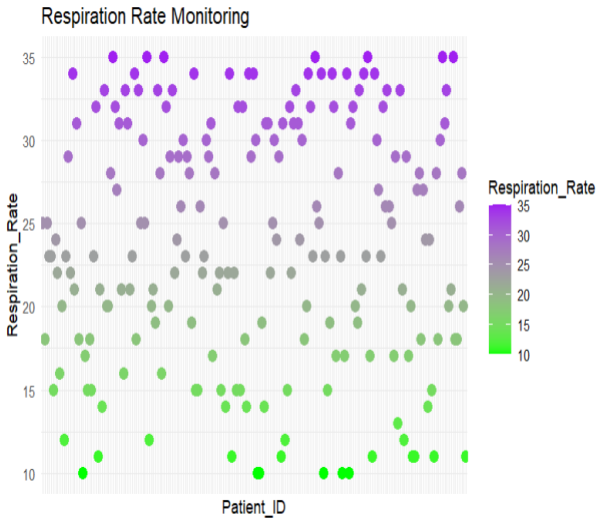
```
  labs(title = "Blood Pressure Monitoring")

ggplot(df, aes(Patient_ID, Respiration_Rate, color = Respiration_Rate)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "green", high = "purple") +
  theme_minimal() +
  theme(axis.text.x = element_blank()) +
  labs(title = "Respiration Rate Monitoring")

ggplot(df, aes(Patient_ID, Temperature, color = Temperature)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "yellow", high = "red") +
  theme_minimal() +
  theme(axis.text.x = element_blank()) +
  labs(title = "Temperature Monitoring")
```

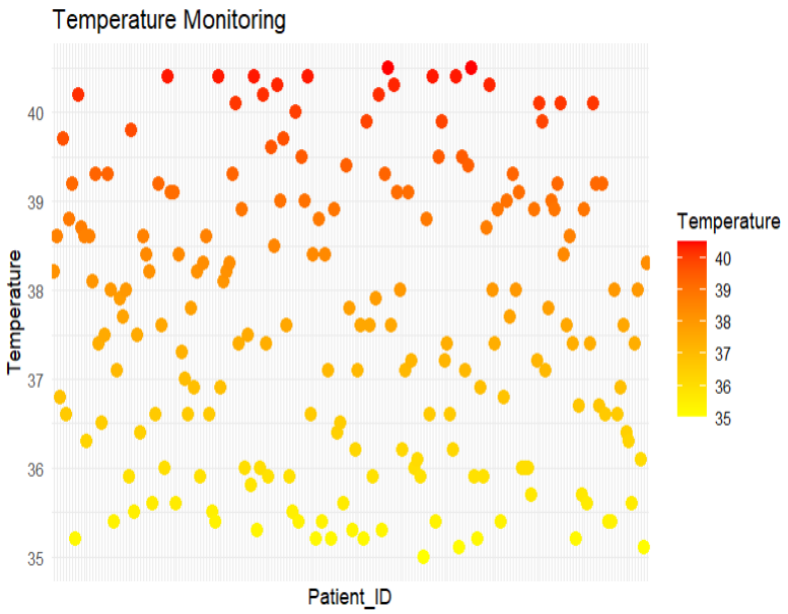
Output :











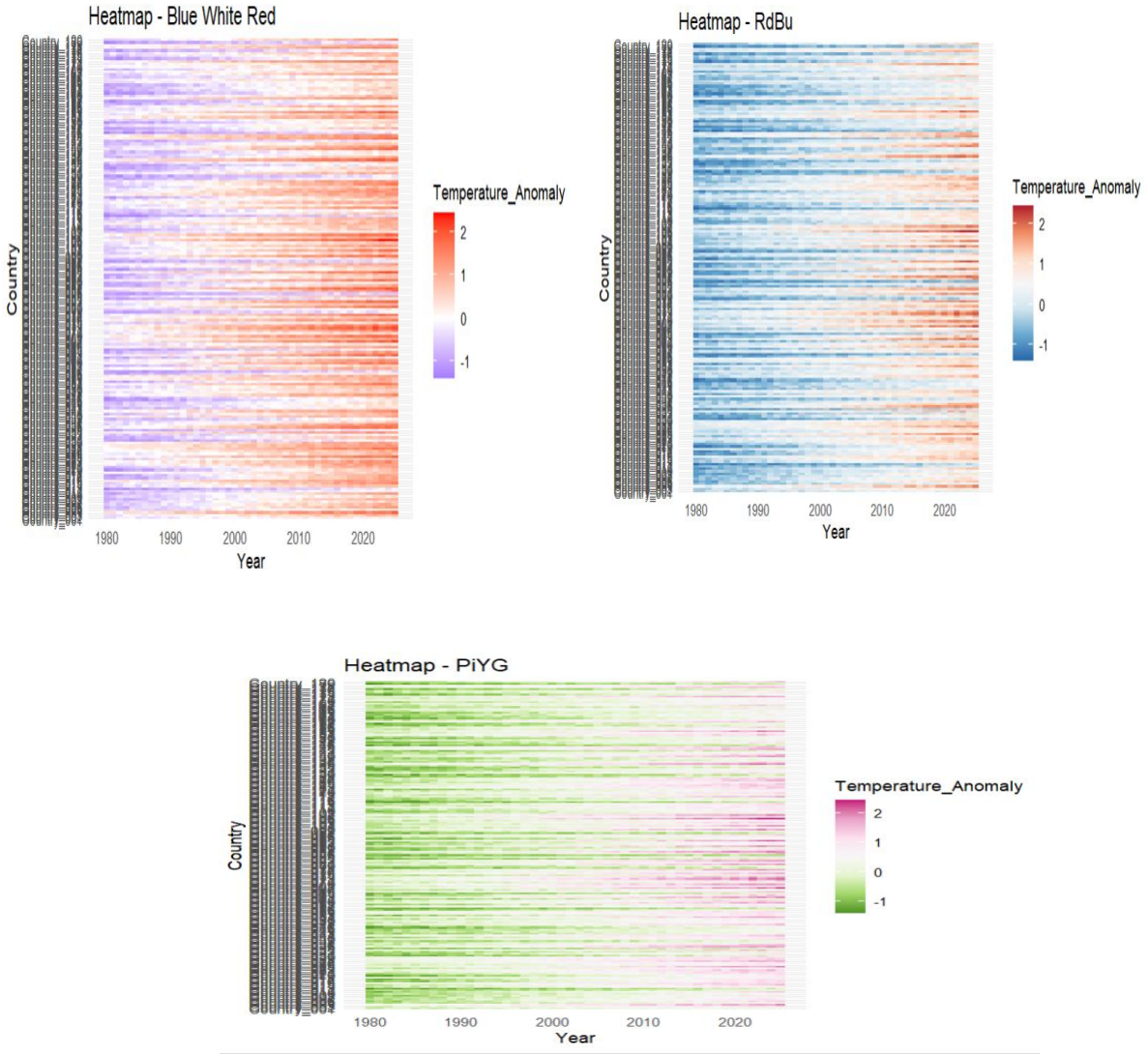
5. **Global Climate Change Study:**

Code :

```
library(ggplot2)

df <-
read.csv("C:/Users/naras/Downloads/DSA0602/global_climate_change_study(1).csv"
)

ggplot(df, aes(x = Year, y = Country, fill = Temperature_Anomaly)) +
  geom_tile() +
```

```
  scale_fill_gradient2(low = "blue", mid = "white", high = "red") +
  labs(title = "Heatmap - Blue White Red") +
  theme_minimal()

ggplot(df, aes(x = Year, y = Country, fill = Temperature_Anomaly)) +
  geom_tile() +
  scale_fill_distiller(palette = "RdBu") +
  labs(title = "Heatmap - RdBu") +
  theme_minimal()

ggplot(df, aes(x = Year, y = Country, fill = Temperature_Anomaly)) +
  geom_tile() +
  scale_fill_distiller(palette = "PiYG") +
  labs(title = "Heatmap - PiYG") +
  theme_minimal()
```

Output :

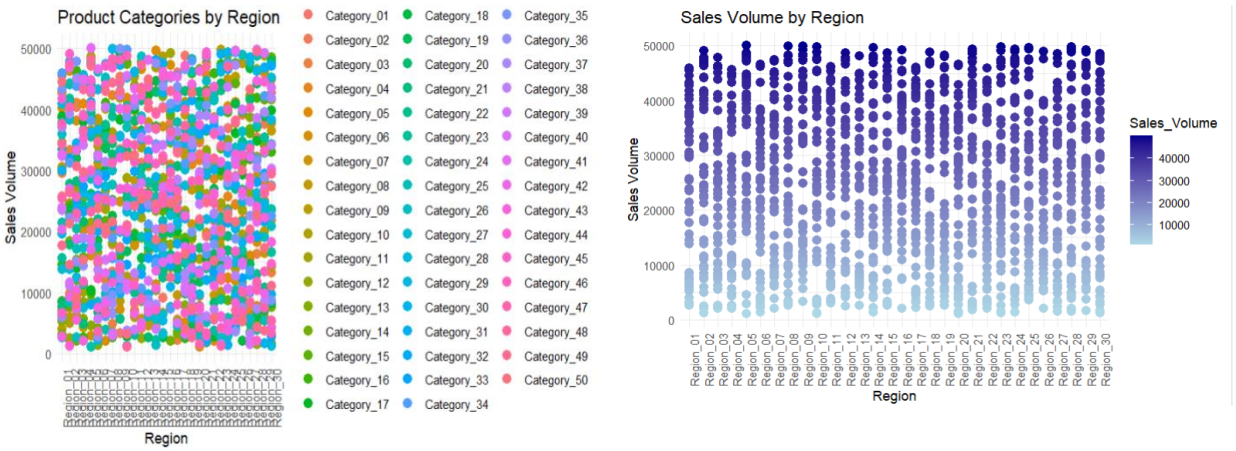


6. E-Commerce Sales Analytics:

Code :

```
library(ggplot2)

df <-
read.csv("C:/Users/naras/Downloads/DSA0602/ecommerce_sales_analytics.csv")

ggplot(df, aes(x = Region, y = Sales_Volume, color = Product_Category)) +
  geom_point(size = 3) +
  labs(
    title = "Product Categories by Region",
    x = "Region",
    y = "Sales Volume"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90))

ggplot(df, aes(x = Region, y = Sales_Volume, color = Sales_Volume)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "lightblue", high = "darkblue") +
  labs(
    title = "Sales Volume by Region",
    x = "Region",
    y = "Sales Volume"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90))
```

Output :

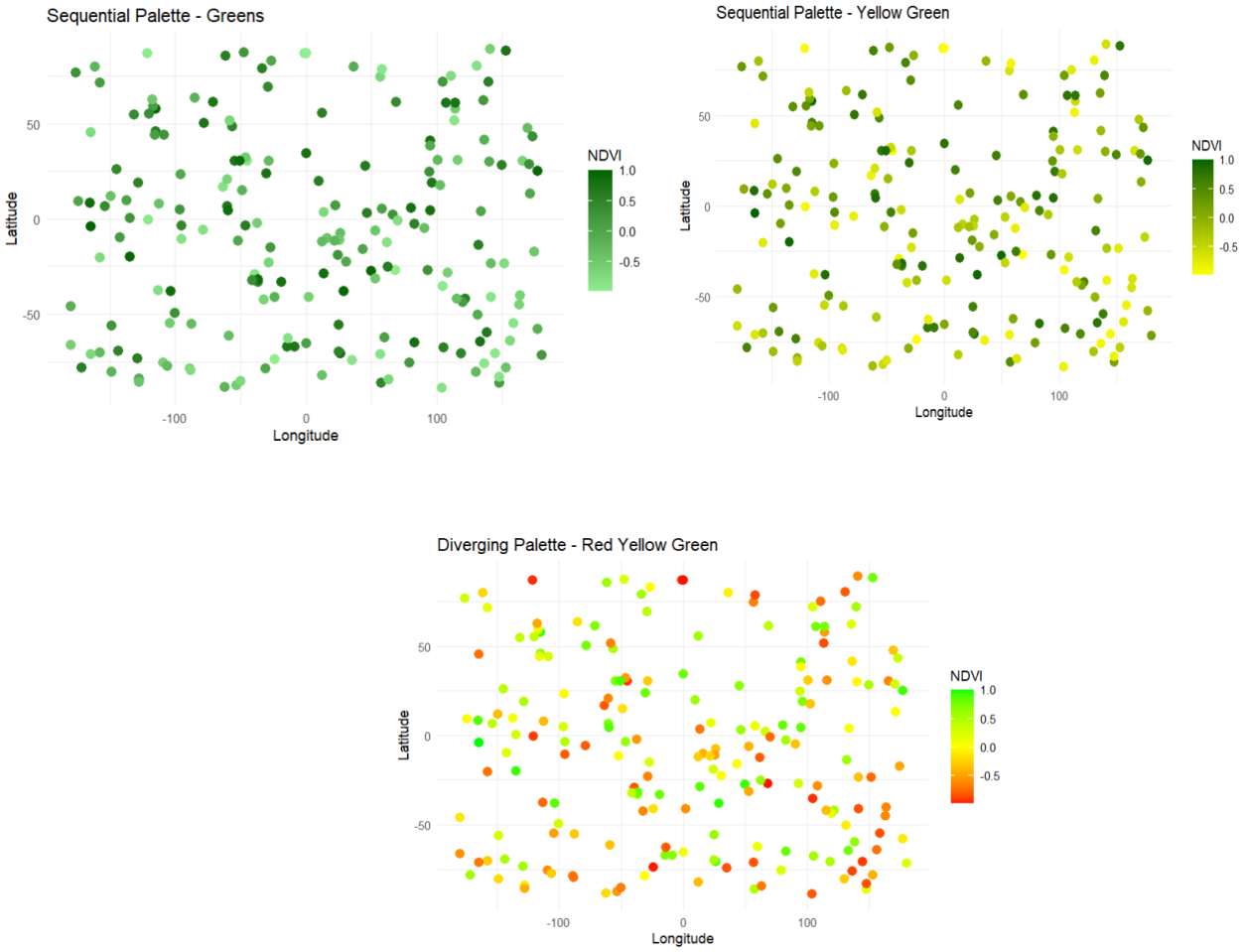


7. **Satellite Image Vegetation Analysis:**

Code :

```
library(ggplot2)

df <-
read.csv("C:/Users/naras/Downloads/DSA0602/satellite_vegetation_analysis.csv")

ggplot(df, aes(x = Longitude, y = Latitude, color = NDVI)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "lightgreen", high = "darkgreen") +
  labs(
    title = "Sequential Palette - Greens",
    x = "Longitude",
    y = "Latitude"
  ) +
  theme_minimal()

ggplot(df, aes(x = Longitude, y = Latitude, color = NDVI)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "yellow", high = "darkgreen") +
  labs(
    title = "Sequential Palette - Yellow Green",
    x = "Longitude",
    y = "Latitude"
  ) +
  theme_minimal()

ggplot(df, aes(x = Longitude, y = Latitude, color = NDVI)) +
  geom_point(size = 3) +
  scale_color_gradient2(
    low = "red",
    mid = "yellow",
    high = "green",
    midpoint = 0
  ) +
  labs(
    title = "Diverging Palette - Red Yellow Green",
    x = "Longitude",
    y = "Latitude"
  ) +
  theme_minimal()
```

Output :



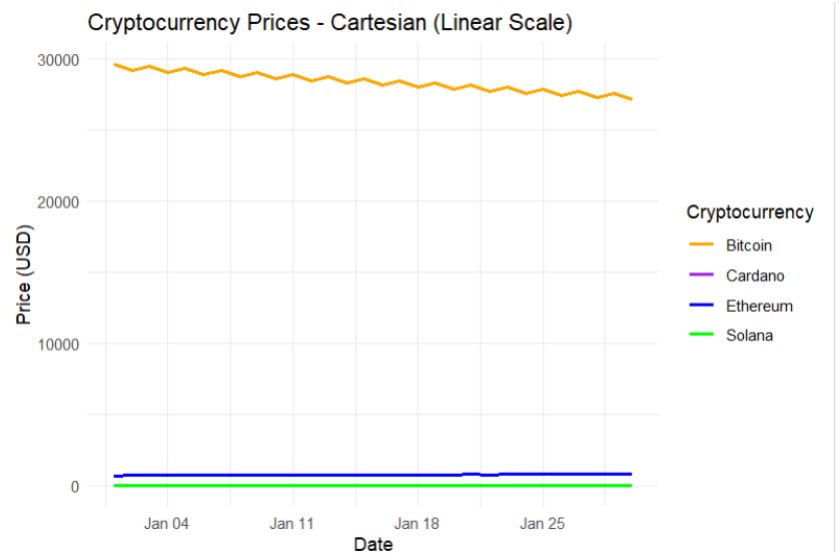
8. **Cryptocurrency Market Volatility:**

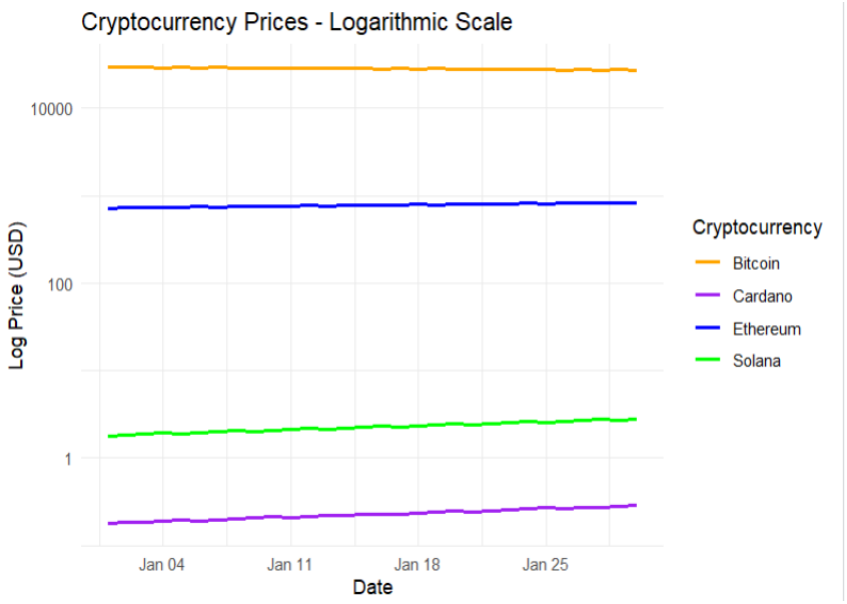
Code :

```
 library(ggplot2)
library(tidyr)

df <-
read.csv("C:/Users/naras/Downloads/DSA0602/cryptocurrency_market_volatility_sample.csv")

df$Date <- as.Date(df$Date)

crypto <- pivot_longer(
  df,
  cols = c(Bitcoin, Ethereum, Solana, Cardano),
  names_to = "Cryptocurrency",
  values_to = "Price"
```

```
)

ggplot(crypto, aes(x = Date, y = Price, color = Cryptocurrency)) +
  geom_line(size = 1) +
  scale_color_manual(values = c(
    "Bitcoin" = "orange",
    "Ethereum" = "blue",
    "Solana" = "green",
    "Cardano" = "purple"
  )) +
  labs(
    title = "Cryptocurrency Prices - Cartesian (Linear Scale)",
    x = "Date",
    y = "Price (USD)"
  ) +
  theme_minimal()

ggplot(crypto, aes(x = Date, y = Price, color = Cryptocurrency)) +
  geom_line(size = 1) +
  scale_y_log10() +
  scale_color_manual(values = c(
    "Bitcoin" = "orange",
    "Ethereum" = "blue",
    "Solana" = "green",
    "Cardano" = "purple"
  )) +
  labs(
    title = "Cryptocurrency Prices - Logarithmic Scale",
    x = "Date",
    y = "Log Price (USD)"
  ) +
  theme_minimal()
```

Output :





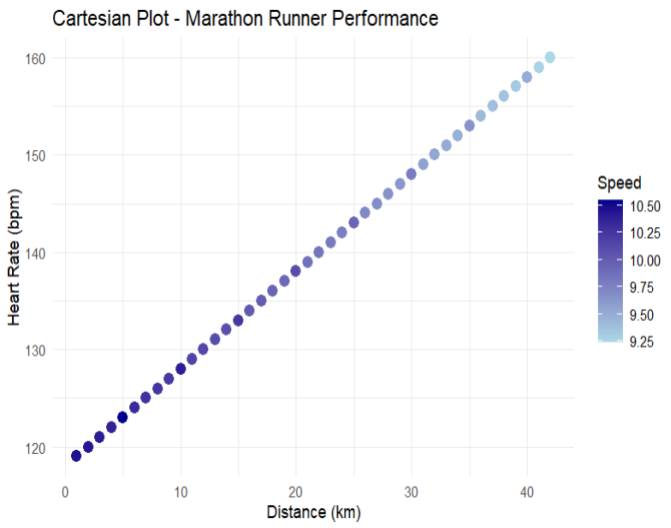
9. **Marathon Runner Performance Analysis:**

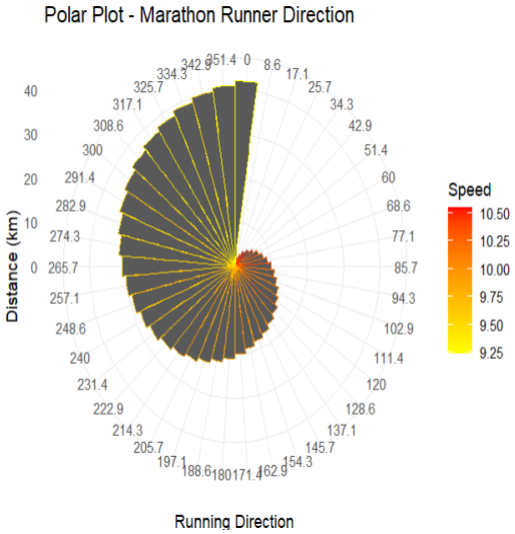
Code :

```
library(ggplot2)

df <-
read.csv("C:/Users/naras/Downloads/DSA0602/marathon_runner_performance.csv")

ggplot(df, aes(x = Distance, y = HeartRate, color = Speed)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "lightblue", high = "darkblue") +
```

```
  labs(
    title = "Cartesian Plot - Marathon Runner Performance",
    x = "Distance (km)",
    y = "Heart Rate (bpm)"
  ) +
  theme_minimal()

ggplot(df, aes(x = factor(RunningDirection), y = Distance, color = Speed)) +
  geom_col(width = 1) +
  coord_polar() +
  scale_color_gradient(low = "yellow", high = "red") +
  labs(
    title = "Polar Plot - Marathon Runner Direction",
    x = "Running Direction",
    y = "Distance (km)"
  ) +
  theme_minimal()
```

Output :





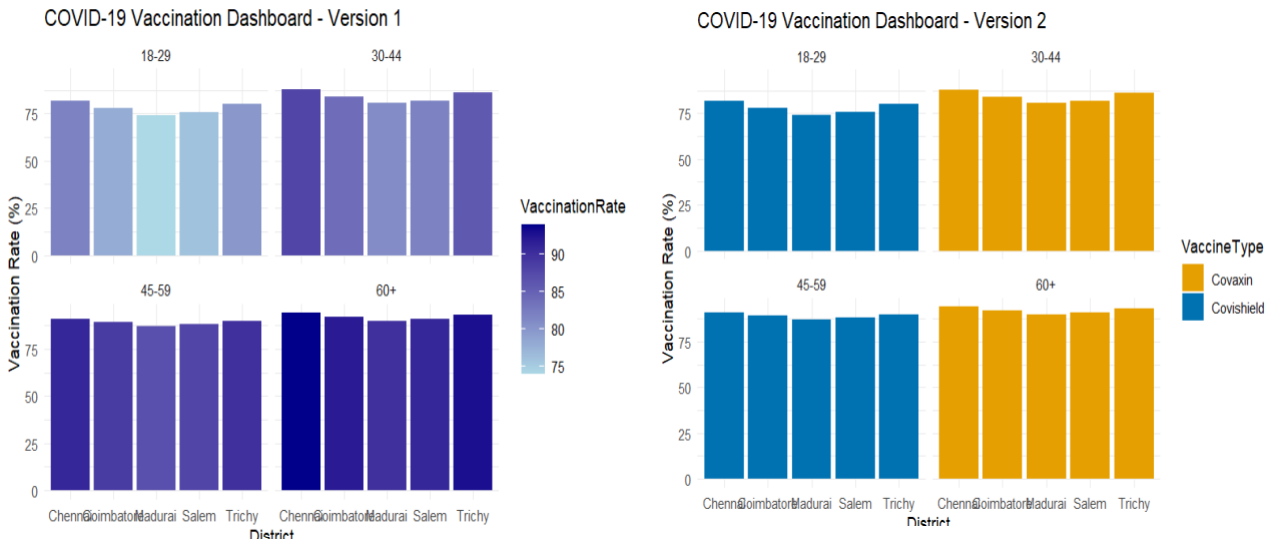
10. COVID-19 Vaccination Campaign Dashboard:

Code :

```
library(ggplot2)

df <- read.csv("C:/Users/naras/Downloads/DSA0602/vaccination_dashboard.csv")
```

```
ggplot(df, aes(x = District, y = VaccinationRate, fill = VaccinationRate)) +
  geom_col() +
  facet_wrap(~AgeGroup) +
  scale_fill_gradient(low = "lightblue", high = "darkblue") +
  labs(
    title = "COVID-19 Vaccination Dashboard - Version 1",
    x = "District",
    y = "Vaccination Rate (%)"
  ) +
  theme_minimal()

ggplot(df, aes(x = District, y = VaccinationRate, fill = VaccineType)) +
  geom_col(position = "dodge") +
  facet_wrap(~AgeGroup) +
  scale_fill_manual(values = c(
    "Covishield" = "#0072B2",
    "Covaxin" = "#E69F00"
  )) +
  labs(
    title = "COVID-19 Vaccination Dashboard - Version 2",
    x = "District",
    y = "Vaccination Rate (%)"
  ) +
  theme_minimal()
```

Output :



Using Python Programming

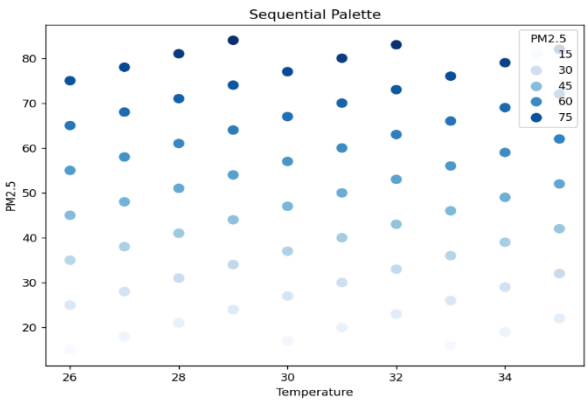
1. **Smart City Air Quality Monitoring:**

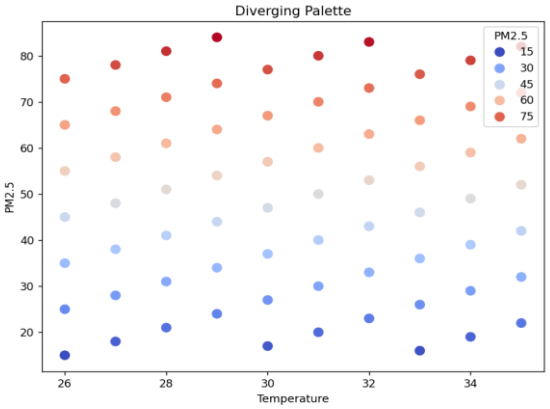
   Code :

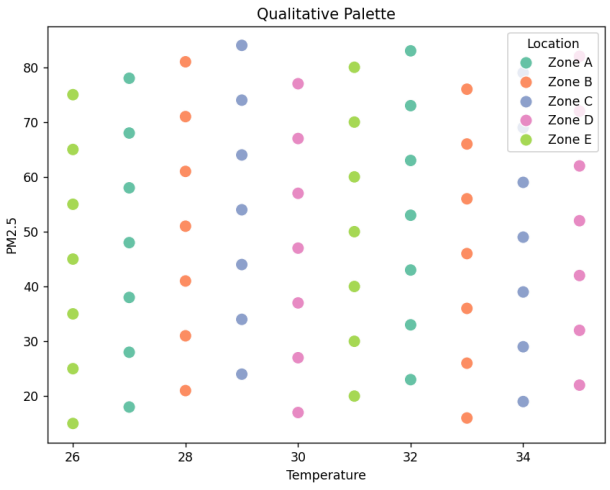
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/air_quality_monitoring_150_sens
ors.csv")

plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="Temperature",
    y="PM2.5",
    hue="PM2.5",
    palette="Blues",
    s=100
)
plt.title("Sequential Palette")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()

plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="Temperature",
    y="PM2.5",
    hue="PM2.5",
    palette="coolwarm",
    s=100
)
plt.title("Diverging Palette")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()
```

```
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="Temperature",
    y="PM2.5",
    hue="Location",
    palette="Set2",
    s=100
)
plt.title("Qualitative Palette")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()
```

Output :





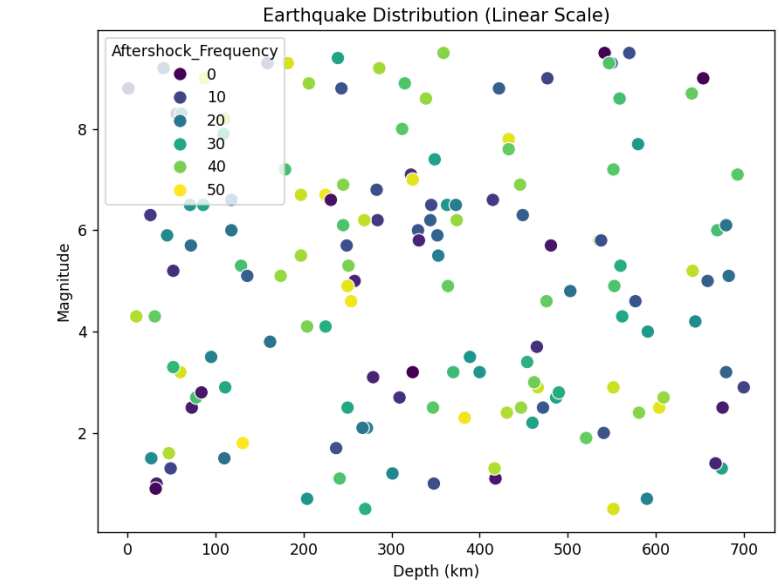


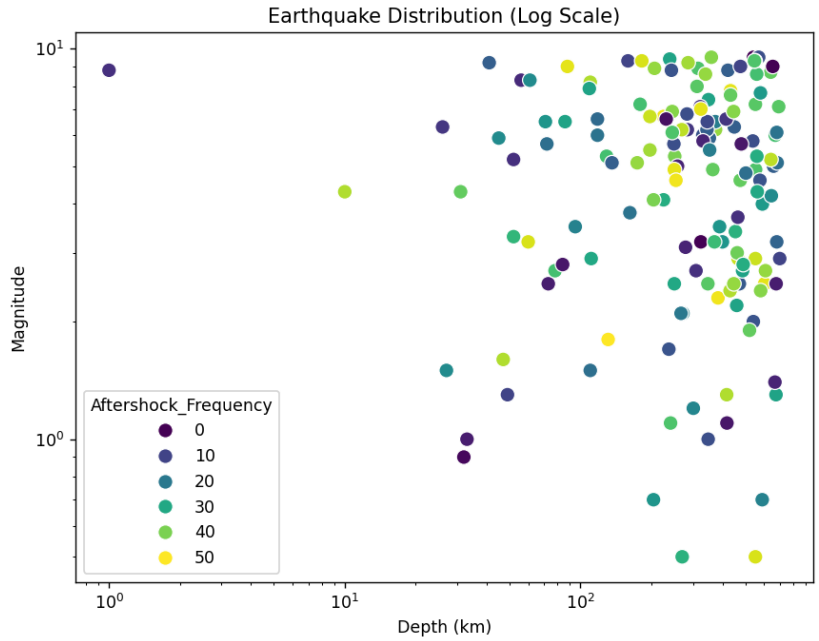
2. **Earthquake Risk Analysis:**

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/earthquake_risk_analysis(2).csv")

plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="Depth",
    y="Magnitude",
    hue="Aftershock_Frequency",
    palette="viridis",
    s=80
)
plt.title("Earthquake Distribution (Linear Scale)")
plt.xlabel("Depth (km)")
plt.ylabel("Magnitude")
plt.show()

plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="Depth",
    y="Magnitude",
    hue="Aftershock_Frequency",
    palette="viridis",
    s=80
)
plt.xscale("log")
plt.yscale("log")
plt.title("Earthquake Distribution (Log Scale)")
plt.xlabel("Depth (km)")
plt.ylabel("Magnitude")
plt.show()
```

Output :





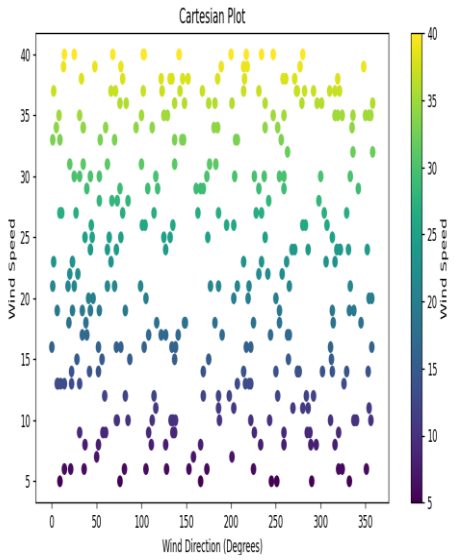
3. Wind Direction Analysis for Airport Operations:

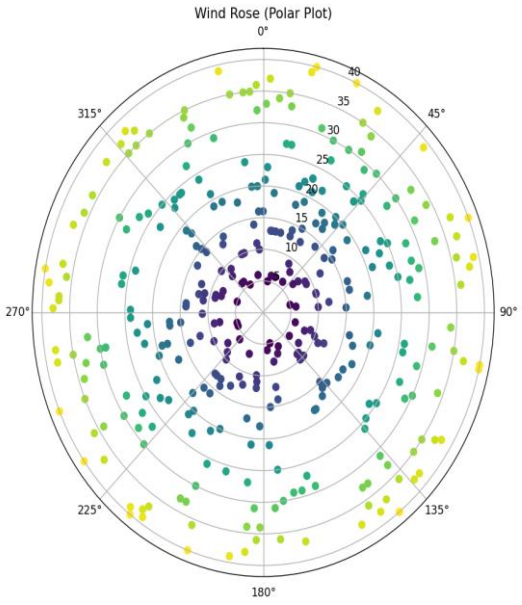
Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/wind_direction_analysis.csv")

plt.figure(figsize=(10,5))
plt.scatter(df["Wind_Direction"], df["Wind_Speed"], c=df["Wind_Speed"],
cmap="viridis")
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed")
plt.title("Cartesian Plot")
plt.colorbar(label="Wind Speed")
plt.show()

theta = np.deg2rad(df["Wind_Direction"])

fig = plt.figure(figsize=(8,8))
ax = plt.subplot(111, projection="polar")
ax.scatter(theta, df["Wind_Speed"], c=df["Wind_Speed"], cmap="viridis")
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
ax.set_title("Wind Rose (Polar Plot)")
plt.show()
```

Output :





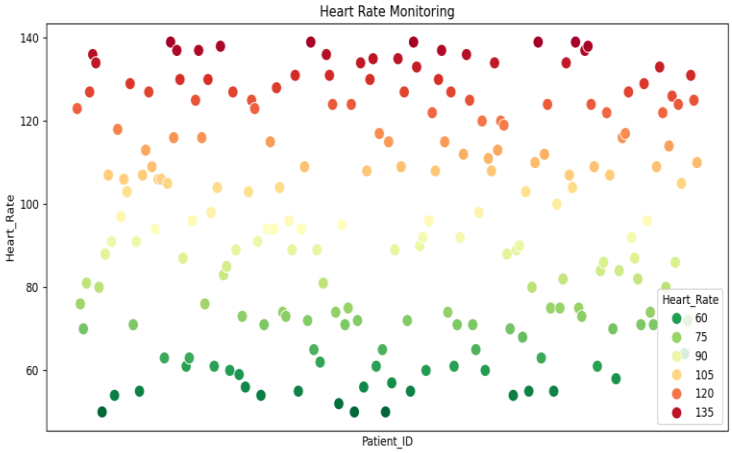
4. **Hospital Patient Monitoring Dashboard:**

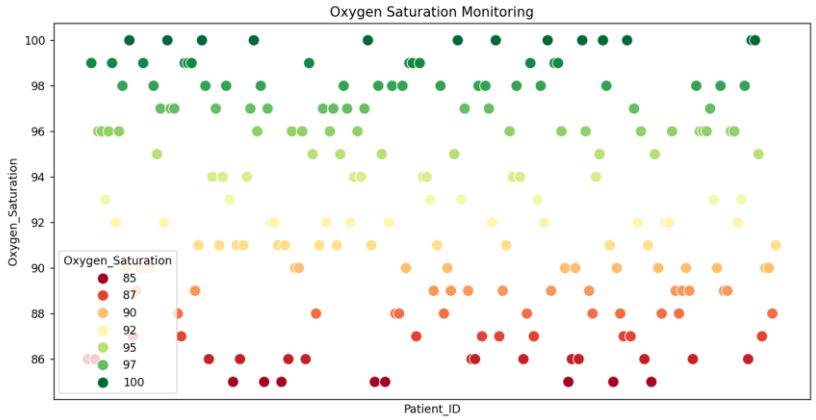
Code :

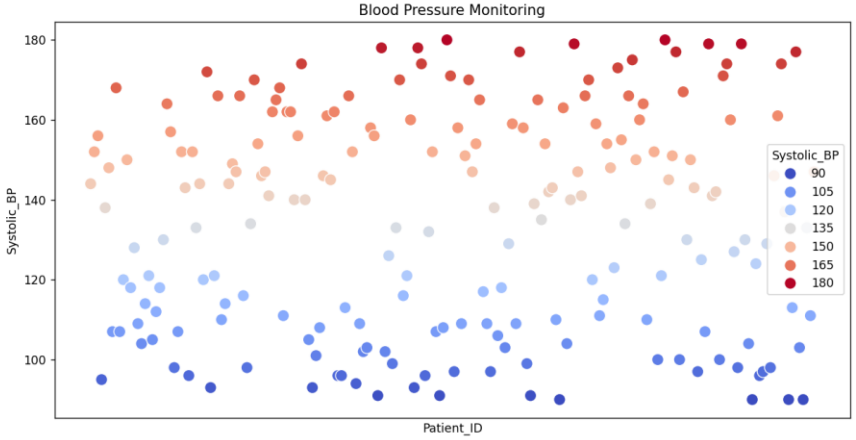
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/hospital_patient_monitoring_dash
board.csv")

plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df,
    x="Patient_ID",
    y="Heart_Rate",
    hue="Heart_Rate",
    palette="RdYlGn_r",
    s=100
)
plt.xticks([])
plt.title("Heart Rate Monitoring")
plt.show()

plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df,
    x="Patient_ID",
    y="Oxygen_Saturation",
    hue="Oxygen_Saturation",
    palette="RdYlGn",
    s=100
)
plt.xticks([])
plt.title("Oxygen Saturation Monitoring")
plt.show()

plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df,
    x="Patient_ID",
    y="Systolic_BP",
    hue="Systolic_BP",
    palette="coolwarm",
    s=100
)
```

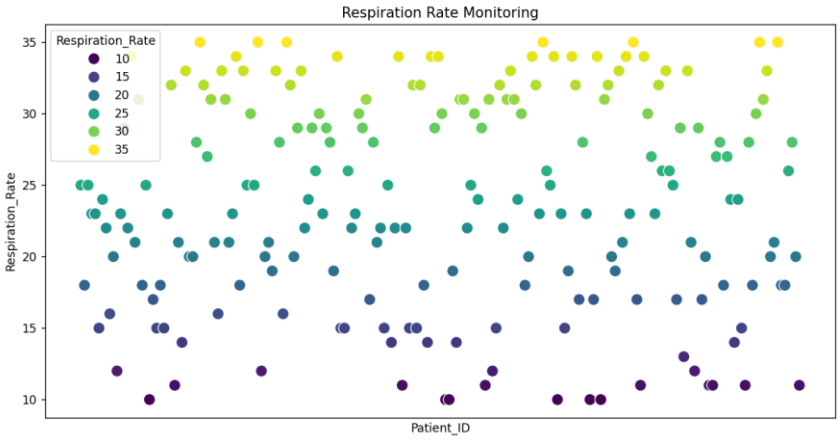
```
plt.xticks([])
plt.title("Blood Pressure Monitoring")
plt.show()

plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df,
    x="Patient_ID",
    y="Respiration_Rate",
    hue="Respiration_Rate",
    palette="viridis",
    s=100
)
plt.xticks([])
plt.title("Respiration Rate Monitoring")
plt.show()

plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df,
    x="Patient_ID",
    y="Temperature",
    hue="Temperature",
    palette="flare",
    s=100
)
plt.xticks([])
plt.title("Temperature Monitoring")
plt.show()
```

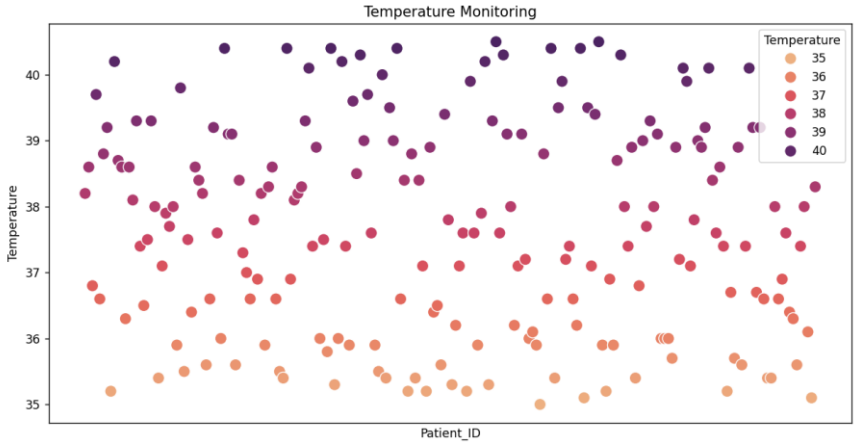
Output :




Oxygen Saturation Monitoring
Oxygen_Saturation
Patient_ID
Oxygen_Saturation
85
87
90
92
95
97
100


Blood Pressure Monitoring
Systolic_BP
Patient_ID
Systolic_BP
90
105
120
135
150
165
180


Respiration Rate Monitoring
Respiration_Rate
Patient_ID
Respiration_Rate
10
15
20
25
30
35

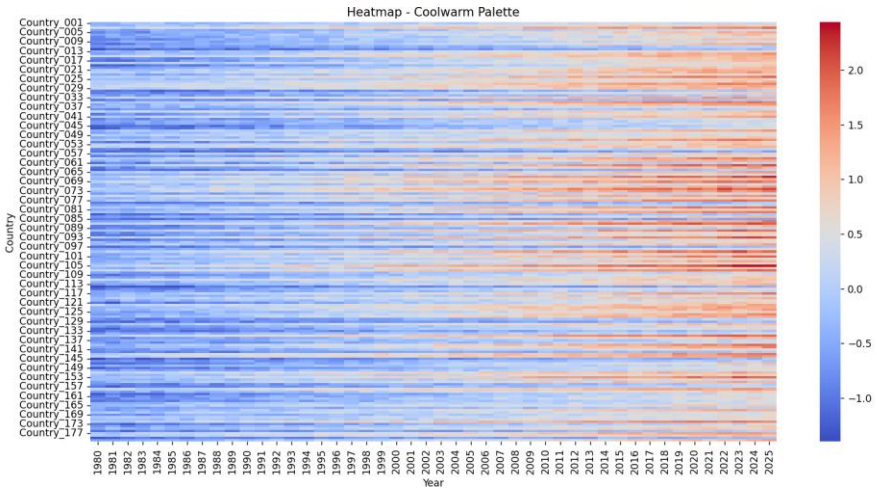

Temperature Monitoring
Temperature
Patient_ID
Temperature
35
36
37
38
39
40

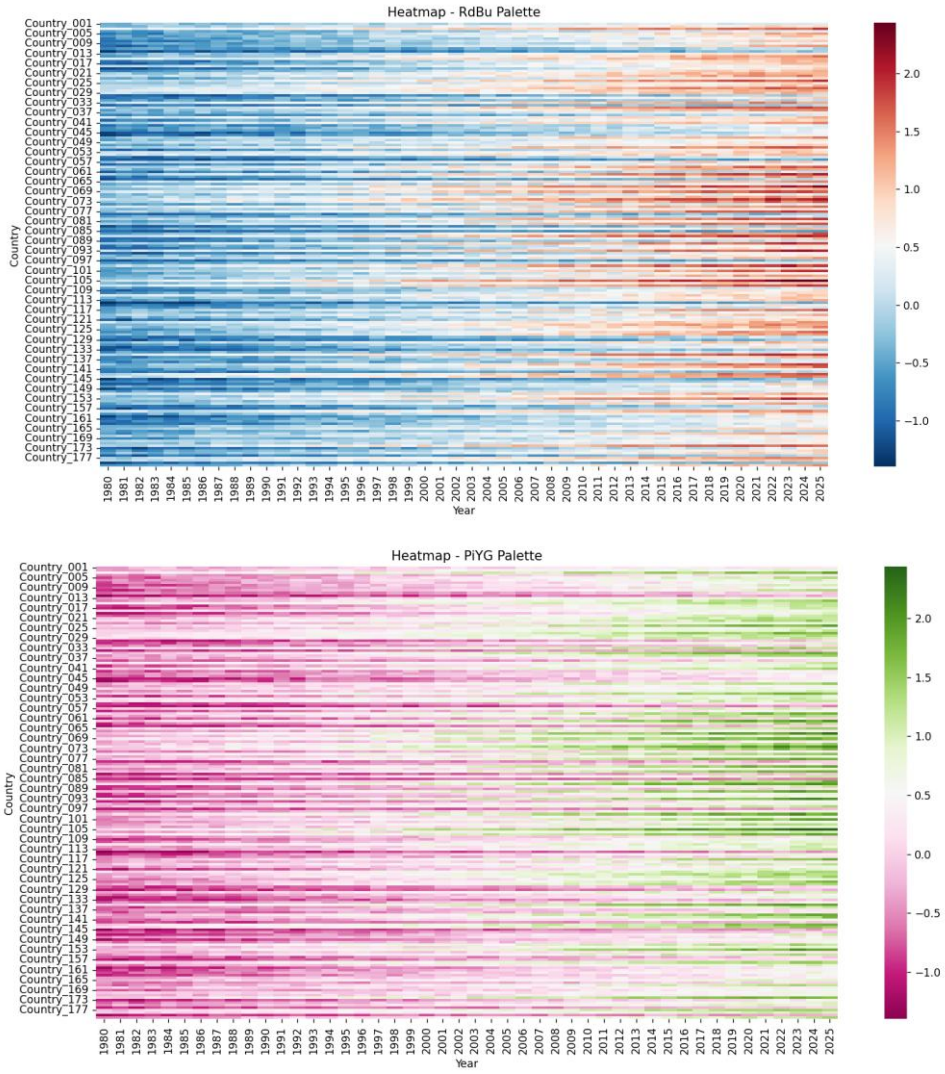
5. **Global Climate Change Study:**

Code :

```
 import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/global_climate_change_study(1).
csv")

heatmap_data = df.pivot(index="Country", columns="Year",
values="Temperature_Anomaly")

plt.figure(figsize=(14,10))
sns.heatmap(heatmap_data, cmap="coolwarm")
plt.title("Heatmap - Coolwarm Palette")
plt.show()

plt.figure(figsize=(14,10))
sns.heatmap(heatmap_data, cmap="RdBu_r")
plt.title("Heatmap - RdBu Palette")
plt.show()

plt.figure(figsize=(14,10))
sns.heatmap(heatmap_data, cmap="PiYG")
plt.title("Heatmap - PiYG Palette")
plt.show()
```

Output :





6. E-Commerce Sales Analytics:

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/ecommerce_sales_analytics.csv")

plt.figure(figsize=(14,8))
sns.scatterplot(
    data=df,
    x="Region",
    y="Sales_Volume",
```

```
    hue="Product_Category",
    palette="tab20",
    s=80
)
plt.xticks(rotation=90)
plt.title("Product Categories by Region")
plt.show()

plt.figure(figsize=(14,8))
sns.scatterplot(
    data=df,
    x="Region",
    y="Sales_Volume",
    hue="Sales_Volume",
    palette="viridis",
    s=80
)
plt.xticks(rotation=90)
plt.title("Sales Volume by Region")
plt.show()
```

Output :

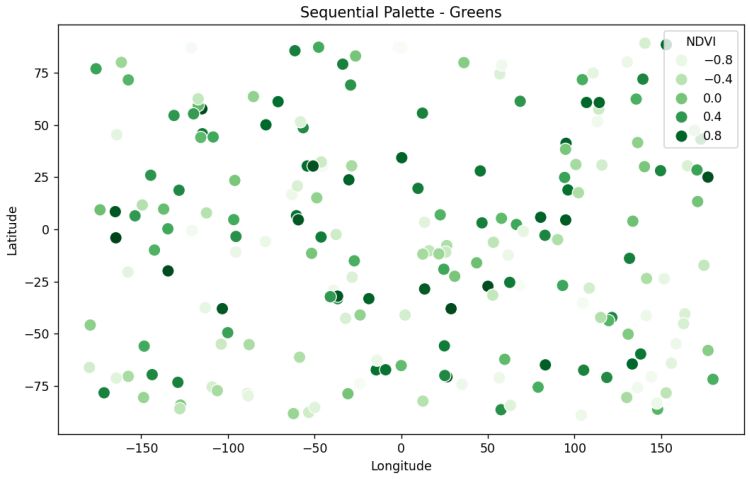
7. **Satellite Image Vegetation Analysis:**

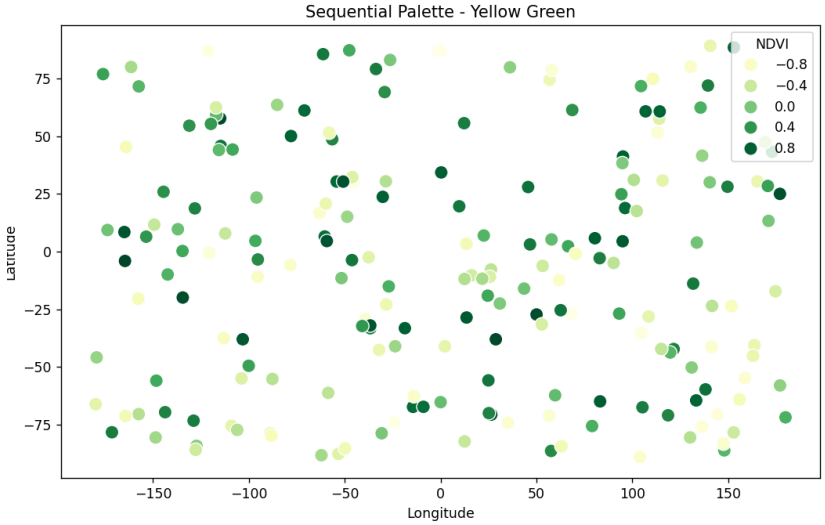
Code :

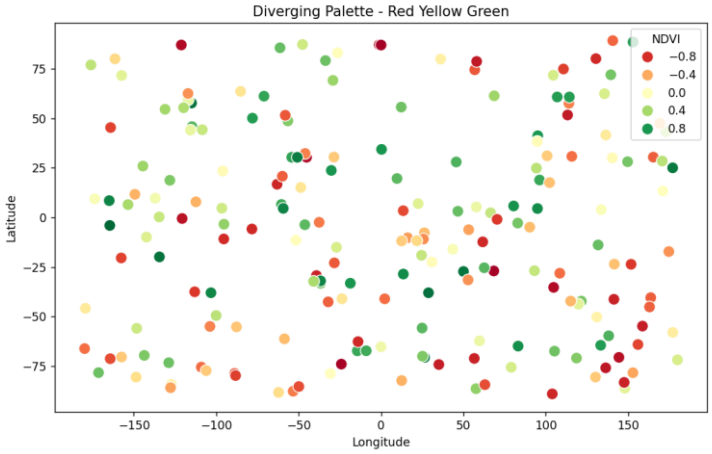
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/satellite_vegetation_analysis.csv"
)

plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
    x="Longitude",
    y="Latitude",
    hue="NDVI",
    palette="Greens",
    s=100
)
plt.title("Sequential Palette - Greens")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()

plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
    x="Longitude",
    y="Latitude",
    hue="NDVI",
    palette="YlGn",
    s=100
)
plt.title("Sequential Palette - Yellow Green")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()

plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
```

```
    x="Longitude",
    y="Latitude",
    hue="NDVI",
    palette="RdYlGn",
    s=100
)
plt.title("Diverging Palette - Red Yellow Green")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()
```

Output :







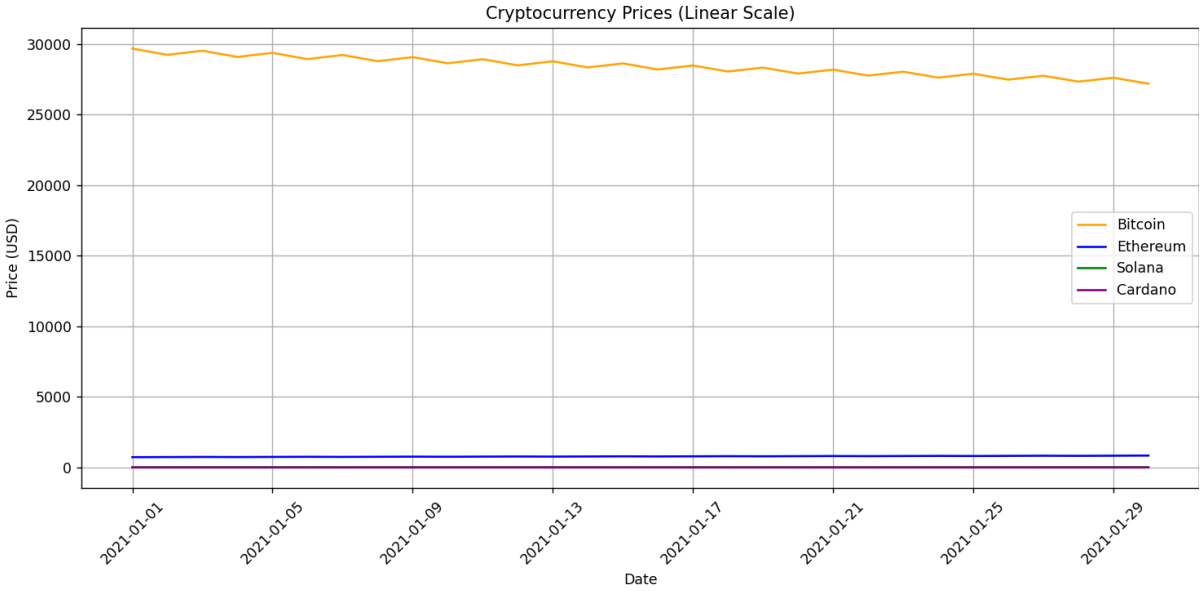
8. **Cryptocurrency Market Volatility:**

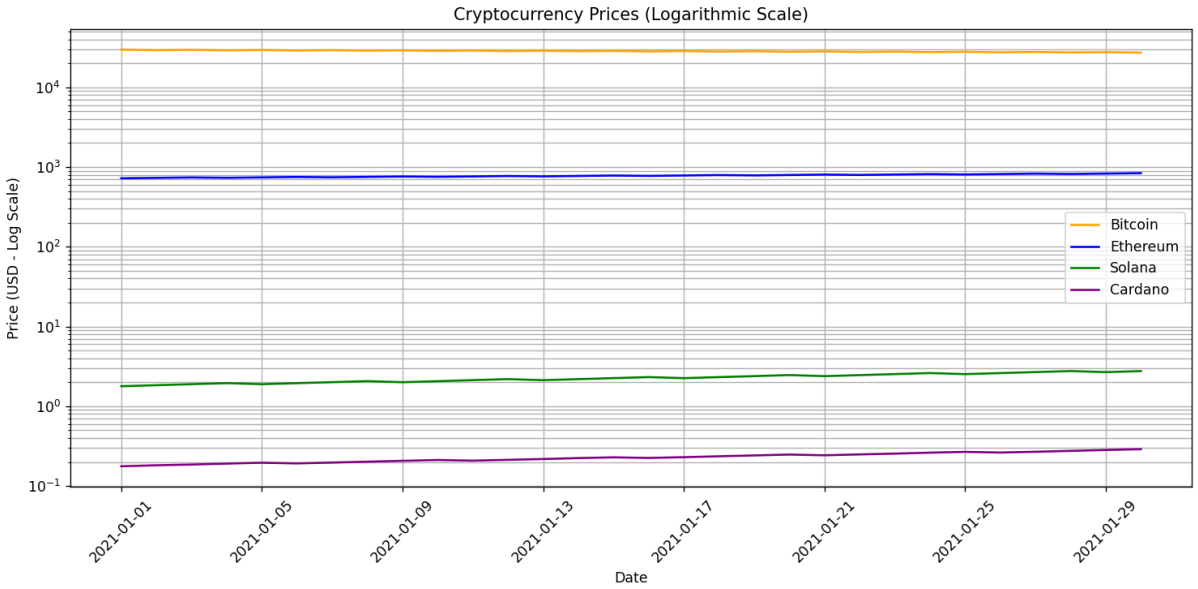
Code :

```
import pandas as pd
import matplotlib.pyplot as plt

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/cryptocurrency_market_volatility
_sample.csv")

df["Date"] = pd.to_datetime(df["Date"])

plt.figure(figsize=(12,6))
plt.plot(df["Date"], df["Bitcoin"], color="orange", label="Bitcoin")
plt.plot(df["Date"], df["Ethereum"], color="blue", label="Ethereum")
plt.plot(df["Date"], df["Solana"], color="green", label="Solana")
plt.plot(df["Date"], df["Cardano"], color="purple", label="Cardano")

plt.title("Cryptocurrency Prices (Linear Scale)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

plt.figure(figsize=(12,6))
plt.plot(df["Date"], df["Bitcoin"], color="orange", label="Bitcoin")
plt.plot(df["Date"], df["Ethereum"], color="blue", label="Ethereum")
plt.plot(df["Date"], df["Solana"], color="green", label="Solana")
```

```
plt.plot(df["Date"], df["Cardano"], color="purple", label="Cardano")

plt.yscale("log")

plt.title("Cryptocurrency Prices (Logarithmic Scale)")
plt.xlabel("Date")
plt.ylabel("Price (USD - Log Scale)")
plt.legend()
plt.grid(True, which="both")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output :



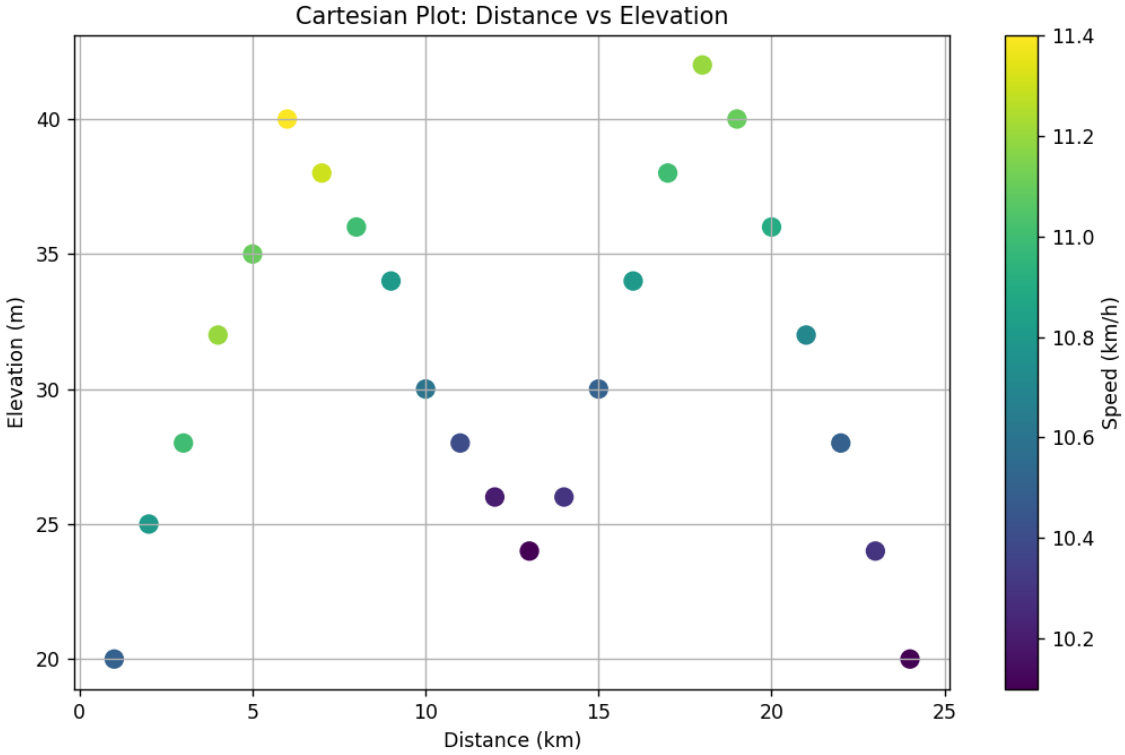


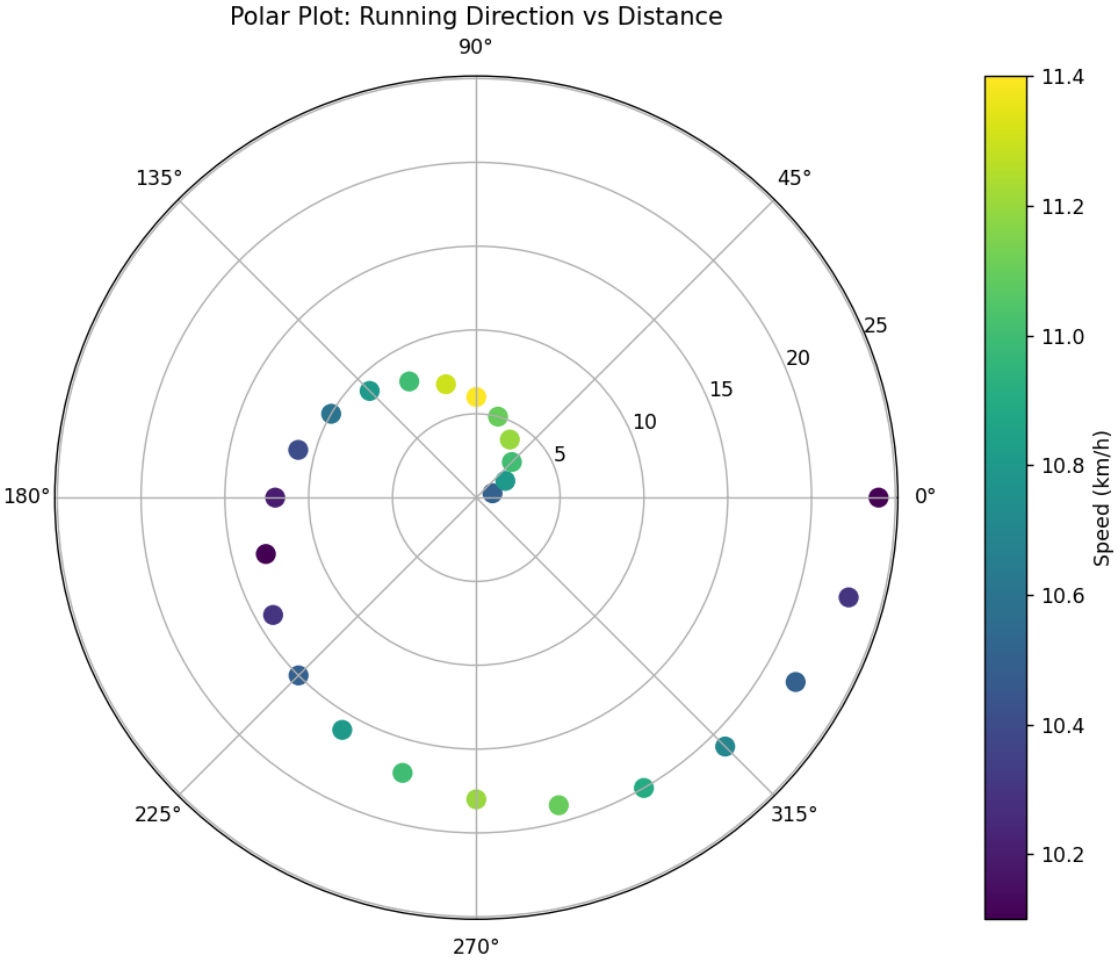
9. **Marathon Runner Performance Analysis:**

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

df = pd.read_csv("C:/Users/naras/Downloads/DSA0602/marathon_runner_data.csv")

df.columns = df.columns.str.strip()

print("Columns in CSV:")
print(df.columns)

if "Direction" in df.columns:
    direction = df["Direction"]
elif "Running Direction" in df.columns:
    direction = df["Running Direction"]
else:
    print("Direction column not found!")
    exit()

plt.figure(figsize=(10,6))
scatter = plt.scatter(df["Distance"], df["Elevation"], c=df["Speed"], cmap="viridis",
s=80)
plt.colorbar(scatter, label="Speed (km/h)")
plt.title("Cartesian Plot: Distance vs Elevation")
plt.xlabel("Distance (km)")
plt.ylabel("Elevation (m)")
plt.grid(True)
plt.show()

theta = np.deg2rad(direction)
r = df["Distance"]

fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(111, projection="polar")
scatter = ax.scatter(theta, r, c=df["Speed"], cmap="viridis", s=80)
plt.colorbar(scatter, label="Speed (km/h)")
ax.set_title("Polar Plot: Running Direction vs Distance")
plt.show()
```

Output :


Cartesian Plot: Distance vs Elevation
Elevation (m)
Distance (km)
Speed (km/h)
40
35
30
25
20
0
5
10
15
20
25
11.4
11.2
11.0
10.8
10.6
10.4
10.2


Polar Plot: Running Direction vs Distance
90°
135°
45°
180°
0°
225°
315°
270°
5
10
15
20
25
Speed (km/h)
11.4
11.2
11.0
10.8
10.6
10.4
10.2

10. COVID-19 Vaccination Campaign Dashboard:

Code :

```
import pandas as pd
import matplotlib.pyplot as plt

df =
pd.read_csv("C:/Users/naras/Downloads/DSA0602/covid_vaccination_dashboard.csv
")

plt.figure(figsize=(10,6))
bars = plt.bar(df["District"], df["VaccinationRate"],
color=plt.cm.Blues(df["VaccinationRate"]/100))
plt.axhline(y=90, color='red', linestyle='--', label='Target (90%)')

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x()+bar.get_width()/2, height+1, f'{height}%', ha='center')

plt.title("COVID-19 Vaccination Dashboard (Policymakers)")
plt.xlabel("District")
plt.ylabel("Vaccination Rate (%)")
plt.legend()
plt.tight_layout()
plt.show()



plt.figure(figsize=(10,6))
bars = plt.bar(df["AgeGroup"], df["Vaccinated"],
color=plt.cm.viridis(df["Vaccinated"]/df["Vaccinated"].max()))

highest = df["Vaccinated"].idxmax()
bars[highest].set_edgecolor("red")
bars[highest].set_linewidth(3)

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x()+bar.get_width()/2, height+100, str(height), ha='center')

plt.title("COVID-19 Vaccination Dashboard (General Public)")
plt.xlabel("Age Group")
plt.ylabel("Vaccinated People")
```

```
plt.tight_layout()
plt.show()
```

Output :